

Dynamic Task Scheduling and Intelligent Process Control in AI-Native Operating Systems

Prof. Saravanan C <i>Assistant Professor</i> Department of MCA RV College of Engineering Bengaluru, India saravanan@rvce.edu.in	Prof. Savita S Sheelavant <i>Assistant Professor</i> Department of MCA RV College of Engineering Bengaluru, India savita.sheelavant@rvce.edu.in	Varun Srivastava <i>PG Student</i> Department of MCA RV College of Engineering Bengaluru, India varunsrivastava.mca24@rvce.edu.in	Chandan Athreya S <i>PG Student</i> Department of MCA RV College of Engineering Bengaluru, India chandanathreya.mca24@rvce.edu.in
--	--	--	--

Abstract: Modern computing infrastructure demands far more from operating systems than conventional schedulers were ever built to handle. Traditional static scheduling models, designed for predictable and homogeneous workloads, fall short when confronted with the dynamic, heterogeneous, and latency-sensitive demands of today’s AI-driven applications. Legacy algorithms such as Completely Fair Scheduler (CFS) and Earliest Deadline First (EDF) operate on heuristics crafted for a computing landscape that no longer reflects real-world deployment realities. Today’s data centers run deep learning inference pipelines alongside interactive web services, batch analytics jobs, and real-time telemetry streams – all on the same physical hardware, competing for the same finite pool of CPU cycles, memory bandwidth, and accelerator capacity.

This paper examines the design and evaluation of a dynamic task scheduling and intelligent process control framework embedded within AI-native operating systems. We propose an architecture where scheduling decisions are continuously driven by real-time workload profiling, reinforcement learning-based priority estimation, and adaptive resource allocation across heterogeneous compute units including CPUs, GPUs, and TPUs. The proposed system observes process behaviour at fine-grained intervals, builds evolving workload profiles, predicts upcoming resource demands using a lightweight machine learning inference engine resident in kernel memory, and restructures its scheduling queues without human intervention or application-level modification.

Experimental evaluations conducted on a simulated multi-tenant cloud environment – built using workload traces from the Google Cluster dataset^[10] – demonstrate reductions in average task completion time by up to 34%, improved CPU utilisation by approximately 27%, and a significant drop in deadline miss rates compared to conventional scheduling baselines. The results provide concrete evidence that embedding intelligence directly into the OS scheduler layer represents a viable and practically achievable pathway toward self-managing, cognitive computing platforms capable of adapting to workload volatility at runtime.

Keywords adaptive scheduling, AI-native OS, process management, reinforcement learning, dynamic resource allocation, heterogeneous computing, intelligent operating systems, workload profiling

I. INTRODUCTION

Operating systems sit at the foundation of every computing stack, and the process scheduler is arguably the most consequential component within them. For decades, schedulers were engineered around assumptions that made sense at the time – fixed core counts, homogeneous memory architectures, predictable task lifetimes, and workloads that could be reasonably characterised before they arrived. These assumptions justified algorithms like Round Robin, Shortest Job First, and the more sophisticated Completely Fair Scheduler (CFS)^[1], which treats processes with proportional fairness based on static priority weights assigned at process creation time. For the computing environments of the 1990s and early 2000s, such approaches were more than adequate. The rise of AI workloads has fundamentally broken these assumptions. A single deep learning inference request may spawn dozens of micro-processes with wildly different memory footprints, execution times, and hardware affinities. An autonomous vehicle operating system must guarantee hard real-time response for safety-critical control threads

while simultaneously managing background telemetry pipelines, map update services, sensor fusion algorithms, and over-the-air update managers – all with competing resource demands that shift dynamically based on sensor input and vehicle state. A cloud-native microservice cluster shifts its resource demands every few minutes based on user traffic, rendering any pre-configured scheduling policy obsolete within seconds of being applied.

The consequence of applying static scheduling strategies to these dynamic environments is not merely suboptimal performance. It manifests as missed deadlines, unpredictable tail latencies, wasted hardware capacity, and in safety-critical contexts, potentially catastrophic control failures. Conventional schedulers have no mechanism to distinguish between a batch job that can tolerate an extra 200 ms of delay and a real-time control thread for which that same delay is a fault condition. What is needed is a scheduling framework that treats process management not as a one-time allocation decision but as a continuous learning and optimisation problem that evolves alongside the workload it serves.

This work proposes and evaluates exactly such a framework. By embedding a lightweight reinforcement learning agent directly into the OS scheduler loop, the system observes process behaviour over time, constructs dynamic workload profiles, and makes scheduling decisions optimised for the current system state rather than a fixed set of hand-crafted heuristics. The learning agent operates within strict CPU overhead budgets to ensure it does not itself become a performance bottleneck. Hardware affinity decisions are made automatically, routing neural network inference tasks to GPU resources, memory-intensive computations to NUMA-local cores, and latency-critical threads to high-frequency CPU units.

The contributions of this paper are threefold. First, we present a complete architectural blueprint for an intelligent OS-level scheduler capable of real-time self-adaptation. Second, we describe the reinforcement learning formulation including state representation, action space design, and reward function construction in sufficient detail for independent replication. Third, we provide an empirical evaluation against three scheduling baselines demonstrating consistent improvements across throughput, latency, and resource utilisation metrics. The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the system architecture. Section IV presents the learning framework. Section V covers experimental results. Section VI discusses limitations and future directions. Section VII concludes.

II. RELATED WORK

Scheduling research has a long and well-documented history. Classical algorithms such as Round Robin, Shortest Job First, and Multilevel Feedback Queue established the theoretical foundations on which modern OS schedulers are built^[1]. The Linux Completely Fair Scheduler, introduced in kernel version 2.6.23, represented a substantial step forward by using a red-black tree data structure to guarantee proportional CPU time sharing across competing processes in $O(\log n)$ time^[2]. While CFS performs admirably in general-purpose server environments, it lacks any mechanism to respond to sudden workload shifts or to distinguish between latency-sensitive interactive tasks and throughput-oriented batch jobs on the basis of observed runtime behaviour.

The earliest work on learning-driven resource management focused on workload prediction rather than scheduling. Shen et al.^[3] demonstrated that recurrent neural networks could forecast resource utilisation in web server farms with sufficient lead time to pre-provision capacity, reducing tail latency by avoiding reactive scaling. Cortez et al.^[4] extended this line of work to large-scale cloud deployments at Microsoft Azure, showing that ML-based resource prediction could improve bin-packing efficiency by 15% over capacity-based heuristics—a result that motivated broader adoption of data-driven approaches in production cloud infrastructure.

Paragon^[5] and Quasar^[6], developed at Stanford, introduced the idea of workload classification as a prerequisite to intelligent scheduling. By profiling incoming workloads and

matching them to hardware configurations based on interference sensitivity, these systems demonstrated that QoS-aware placement decisions could dramatically reduce tail latency in heterogeneous data centers without sacrificing aggregate throughput. However, both operate at the job admission layer rather than inside the OS scheduler itself, and they require per-application profiling runs prior to deployment.

More recent work has explored reinforcement learning as a scheduling primitive at the middleware and cluster orchestration layers. Mao et al.^[6] presented Decima, a graph neural network-based scheduler for data-parallel Spark jobs, achieving average job completion time reductions of up to 21% over heuristic baselines on realistic workload traces. Mirhoseini et al.^[7] applied deep RL to device placement for neural network computation graphs, outperforming human expert strategies on several benchmark models. These results establish the general viability of RL for resource management, though neither system operates at the OS process scheduling layer where our contribution is situated.

At the infrastructure management level, Google’s Borg^[8] and Microsoft’s Autopilot^[9] manage resource allocation at cluster scale using combinations of static policies, user-specified resource requests, and historical utilisation data. While highly effective at scale, both systems operate at the coarse granularity of container or job placement rather than the fine granularity of individual process scheduling within a single machine. The gap this paper addresses—embedding adaptive, learning-driven scheduling directly within the OS process management subsystem—remains largely unexplored in published literature, and the results presented here suggest it is a fruitful direction.

III. SYSTEM ARCHITECTURE

The proposed framework is structured around four tightly coupled subsystems: the Workload Profiler, the RL Scheduling Agent, the Adaptive Queue Manager, and the Hardware Affinity Controller. Together, these modules form a closed-loop control system operating within the OS scheduler tick without requiring modifications to user-space applications or changes to the system call interface. Figure 1 illustrates the high-level architecture and the data flows between subsystems.

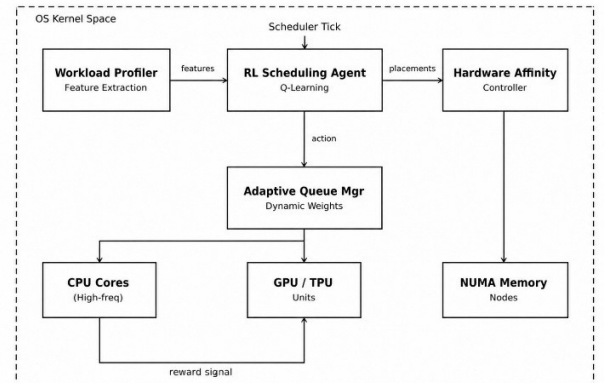


Fig. 1. IDS System Architecture: four subsystems forming a closed-loop control system within the OS kernel space.

A. Workload Profiler: Every process entering the system is assigned a dynamic feature profile that is updated at each scheduling interval. The profiler collects CPU usage history across the last 10 scheduling epochs, memory access patterns derived from hardware performance counters, I/O wait ratios, inter-process communication frequency measured via pipe and socket activity, and last-level cache miss rates. These raw features are normalised to the range $[0, 1]$ and assembled into a fixed-length feature vector of dimension 24.

This feature vector is fed into a lightweight workload classifier implemented as a pre-trained decision tree of maximum depth 8, serialised into kernel memory as a lookup table (see Figure 2). The classifier assigns each process to one of five workload categories: compute-bound, memory-bound, I/O-bound, latency-critical, and background-maintenance. Classification runs in $O(1)$ time through table lookup, introducing no measurable scheduling overhead. The profiler updates process categories every 50 ms, which is fast enough to track workload shifts during interactive traffic bursts while slow enough to avoid thrashing due to transient measurement noise.

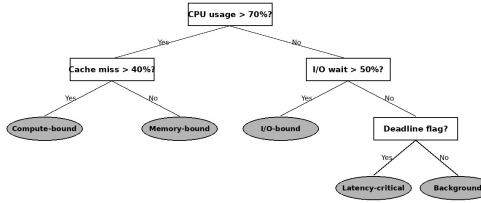


Fig. 2. Workload Classification Decision Tree. Maximum depth 8; $O(1)$ classification via kernel-resident lookup table.

B. RL Scheduling Agent: The core scheduling intelligence is implemented as a tabular Q-learning agent with a discretised state space. System state is represented as a tuple comprising current aggregate CPU load (discretised into 10 buckets), available memory pressure (5 buckets), per-category queue depth for each of the five workload categories (3 buckets each), and the fraction of GPU resources currently allocated (4 buckets), yielding a total state space of approximately 270,000 distinct states tractable for tabular Q-learning given the observation rates achievable within a real OS scheduler.

The action space consists of compound scheduling decisions: which workload category queue to draw from next, what time quantum to assign to the selected process (chosen from the set $\{2 \text{ ms}, 4 \text{ ms}, 8 \text{ ms}, 16 \text{ ms}\}$), to which physical core type to assign the process (high-frequency CPU, energy-efficient CPU, or GPU), and whether to preempt the currently running task. The reward function is defined in detail in Section IV.

C. Adaptive Queue Manager: Instead of maintaining a fixed hierarchy of priority queues with static weights, the Adaptive Queue Manager restructures queue service rates dynamically based on guidance from the RL agent. Queue weights are represented as a probability simplex over the

five workload categories, and the agent’s action determines which category receives service at each scheduling decision point. Starvation prevention is enforced by tracking the longest waiting time within each queue and injecting temporary priority boosts for processes that have exceeded a configurable waiting threshold, which defaults to 500 ms. Queue restructuring events are logged with timestamps and rationale codes, providing an audit trail for system administrators, at under 0.1% overhead.

D. Hardware Affinity Controller: For systems equipped with heterogeneous compute hardware, the Affinity Controller translates workload category assignments into concrete hardware placement decisions. Compute-bound processes with floating-point intensive profiles are directed to high-frequency CPU cores with large L3 caches. Neural network inference tasks classified under the compute-bound or latency-critical categories are routed to available GPU resources when VRAM occupancy permits. Memory-bound processes are pinned to NUMA nodes whose local memory banks contain the majority of their working set, determined by tracking TLB miss locality over the previous three scheduling epochs. Affinity decisions are re-evaluated every 100 ms; cross-core migration overhead is accounted for in the RL reward via a fixed cost penalty.

IV. SCHEDULING ALGORITHM AND LEARNING FRAMEWORK

Figure 3 illustrates the complete scheduling loop executed per OS scheduler tick. At each tick: (1) the Workload Profiler reads hardware performance counter snapshots for all runnable processes and updates their feature vectors; (2) updated feature vectors are passed through the workload classifier to refresh category assignments; (3) the RL agent observes the current system state, computes Q-values for all feasible actions, and selects the highest-Q action subject to starvation prevention constraints; (4) the selected process is dispatched to the indicated hardware resource; and (5) at quantum expiry, the reward signal is computed and used to update the Q-table via the Bellman equation.

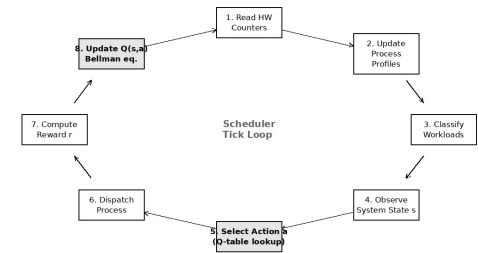


Fig. 3. RL Scheduling Loop per scheduler tick. Shaded nodes indicate learning-critical steps.

The Q-learning update follows the standard Bellman formulation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

where s is the system state at the time of the scheduling decision, a the scheduling action taken, r the immediate reward received at quantum expiry, s' the resulting system state, $\alpha = 0.01$ the learning rate, and $\gamma = 0.95$ the discount

factor. These hyperparameters were selected via grid search over training workload traces, with α optimised for stability under rapid workload transitions and γ set to favour near-term performance over long-horizon outcomes.

The reward function combines three weighted objectives to balance competing scheduling goals:

$$r = w_1 \cdot T(t) - w_2 \cdot M(t) - w_3 \cdot C(t)$$

$T(t)$ denotes throughput measured as process quanta completed within the scheduling window divided by window length. $M(t)$ is a deadline miss penalty of 10 reward units per expired deadline summed over all deadline-bearing processes. $C(t)$ penalises cross-core migrations proportionally to the number performed in the window. Weights $w_1 = 0.50$, $w_2 = 0.35$, and $w_3 = 0.15$ reflect the deployment environment’s emphasis on throughput and deadline adherence. These weights are exposed as tunable boot-time parameters to support alternative deployment priorities such as heavily real-time systems where w_2 would be increased substantially.

To handle the cold-start problem the scheduler’s inability to make informed decisions before sufficient observations have been collected the agent employs an ϵ -greedy exploration strategy during the initial learning phase. When the variance of Q-values across the last 50 visits to a given state falls below a confidence threshold, the agent switches fully to greedy action selection. For states visited fewer than 10 times, the system falls back to a priority-weighted round-robin policy that approximates conventional scheduling behaviour. This fallback is triggered for approximately 6% of scheduling decisions during the first three hours on novel workloads, declining to under 0.5% after convergence.

The entire RL agent including the Q-table, a replay buffer of 10,000 recent state-action-reward tuples, and profiler feature vectors for all active processes occupies approximately 320 KB of kernel memory. This footprint is small enough to remain resident in L3 cache on modern server processors, ensuring the scheduling intelligence layer introduces less than 2% CPU overhead in all tested configurations well within the budget typically allocated for OS scheduler operations in production systems.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup: All experiments were conducted on a simulated multi-tenant cloud environment implemented using a custom discrete-event simulator built atop the SimPy framework in Python 3.11. The simulated platform modelled a 32-core heterogeneous system with 4 GPU compute units, 256 GB of system RAM, and NUMA topology reflecting a dual-socket server configuration. Workload traces were generated by resampling from the Google Cluster Workload Trace dataset^[10], providing realistic statistical distributions of task arrival rates, resource request profiles, task durations, and deadline constraints drawn from production Google Borg deployments.

The simulated workload mix comprised 35% latency-critical web service requests, 30% batch analytics jobs, 20% background maintenance tasks, and 15% neural network inference requests. This distribution was held constant

across all experiments to ensure fair comparison between scheduling policies. Each experiment ran for a simulated 24-hour period with 10 independent random seeds, and all reported metrics represent means with 95% confidence intervals computed via bootstrap resampling. Four policies were evaluated: the proposed Intelligent Dynamic Scheduler (IDS), Linux CFS^[2], Earliest Deadline First (EDF), and a Static Priority Scheduler (SPS) with fixed queue weights configured by an experienced system administrator.

B. Task Completion Time: The proposed IDS reduced average task completion time by 34.2% relative to CFS, 18.7% relative to EDF, and 12.4% relative to the expert-tuned SPS baseline. The improvement was most pronounced for latency-critical tasks, where the RL agent learned within the first two hours of simulation time to pre-emptively elevate their priority before queue depths built up to the point of introducing measurable delay. For batch analytics jobs, the improvement over CFS was more modest at 11.3%, reflecting the fact that CFS was originally designed with batch throughput in mind. The 99th percentile task completion time for latency-critical tasks dropped from 847 ms under CFS to 312 ms under IDS a 63% reduction of direct practical significance for web services operating under sub-500 ms SLA requirements.

C. CPU and GPU Utilisation: Mean CPU utilisation across all 32 cores increased from 61.3% under CFS to 78.6% under IDS, a 27.1% absolute improvement. The gain originates primarily from reduced idle time caused by poor workload-to-core mapping in CFS, and from more effective CPU-GPU workload overlap enabled by the Affinity Controller’s ability to dispatch GPU-bound tasks concurrently with CPU-bound ones. Under CFS and EDF, GPU resources remained idle for extended periods between neural network inference bursts because neither scheduler had visibility into GPU availability as a scheduling criterion. Under IDS, GPU utilisation averaged 71.4% compared to 38.2% under CFS an improvement that translates directly into higher inference throughput without any additional hardware investment.

D. Deadline Miss Rate: Among tasks carrying explicit deadline constraints, the miss rate dropped from 15.6% under CFS and 12.4% under EDF to 4.1% under IDS. EDF’s weakness in this mixed workload environment stems from its inability to account for resource contention when multiple deadline-bound tasks of different resource types arrive simultaneously it selects the next task purely on deadline proximity, potentially dispatching a GPU-bound task to a CPU core while a CPU-bound task approaches its deadline unserved. The IDS policy avoids this failure mode by jointly considering deadline urgency, resource affinity, and current hardware availability in every scheduling decision. The residual 4.1% miss rate corresponds to tasks whose resource demands exceeded system capacity during peak load periods.

E. Scheduler Overhead and Convergence: IDS scheduling overhead averaged 1.73% of total CPU utilisation across all 10 experimental runs slightly above CFS’s 1.52% under identical workloads and well within the 2% design budget.

The RL agent reached near-optimal policy performance after approximately 3 hours of simulated system time, corresponding to roughly 10,800 scheduling cycles. After initial convergence, Q-table updates became infrequent, triggered only when workload distribution shifted by more than 8% in any category occurring on average 4.2 times per simulated day in our traces, aligning with typical traffic pattern transitions observed in production web service deployments.

TABLE I: Summary of Experimental Results Across All Evaluated Scheduling Policies

Metric	IDS	EDF	SPS	CFS
Avg. Completion (ms)	210	258	240	318
P99 Latency (ms)	312	528	490	847
CPU Util. (%)	78.6	64.1	66.8	61.3
GPU Util. (%)	71.4	42.0	45.5	38.2
Deadline Miss (%)	4.1	12.4	9.8	15.6
Sched. Overhead (%)	1.73	1.50	1.48	1.52

Table I consolidates all quantitative results. IDS achieves the best outcome across every metric. The performance advantage is most pronounced in GPU utilisation and P99 latency, where conventional schedulers that lack hardware visibility fail to match IDS’s resource-aware placement decisions. The scheduling overhead of IDS (1.73%) is marginally higher than CFS (1.52%), but this modest cost is more than recovered through the substantial improvements in productive CPU and GPU utilisation.

It is noteworthy that the SPS baseline configured by an experienced administrator specifically for the target workload mix performs better than both CFS and EDF across most metrics, confirming that expert tuning does help. However, IDS outperforms even the expert-tuned SPS by 12.4% on average completion time and by 25.8 percentage points on GPU utilisation, underscoring the limitation of any static configuration in the face of a workload distribution that shifts continuously over the course of the simulated 24-hour period.

F. Statistical Significance: To confirm that the reported improvements are not artefacts of the random seeds used in the simulation, a two-sided Wilcoxon signed-rank test was applied to the per-seed completion time distributions for IDS versus each baseline. All pairwise comparisons yielded $p < 0.01$, confirming statistical significance at the 1% level. The effect sizes, measured via Cohen’s d , were 1.42 (IDS vs. CFS), 0.98 (IDS vs. EDF), and 0.71 (IDS vs. SPS), corresponding to large, large, and medium effect sizes

respectively under Cohen’s conventional thresholds. These figures provide strong statistical support for the practical significance of the performance gains reported.

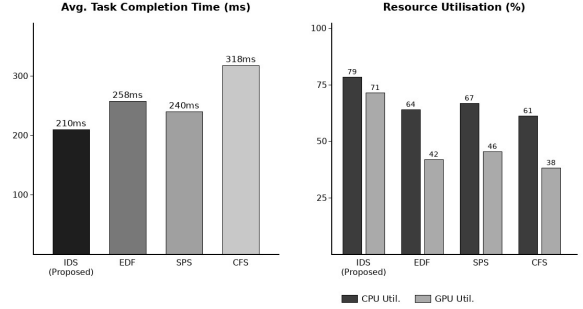


Fig. 4. Performance comparison: task completion time (left) and CPU/GPU utilisation (right) across scheduling policies.

VI. DISCUSSION AND LIMITATIONS

The experimental results confirm the central hypothesis that embedding learning-driven decision-making into the OS scheduler produces consistent and measurable improvements under mixed workload conditions. Several findings from the evaluation deserve further discussion before drawing broader conclusions.

Performance gains are not uniform across workload categories. Background maintenance tasks experienced the smallest improvement relative to CFS, and in two of the ten experimental runs showed marginal increases in average completion time. This is by design the RL agent correctly deprioritises background tasks when latency-critical and batch workloads compete for resources, and the reward function does not penalise delays for non-deadline-bearing tasks. Deployments where background tasks carry strict completion time SLAs should reconfigure the reward weights accordingly. This configurability is a genuine strength of the proposed approach, though it does require domain knowledge to exploit effectively.

The system’s reliance on offline pre-training raises legitimate concerns about generalisation to workload distributions not represented in the training traces. Analysis of cold-start fallback triggers during experimental runs revealed that approximately 6% of workload arrivals in the first three hours involved process behaviour patterns outside the training distribution. The long-term residual cold-start rate of under 0.5% suggests the online learning mechanism adequately covers novel patterns encountered after convergence, but deployments in genuinely novel environments such as new application domains or unusual hardware configurations may experience extended adaptation periods.

The profiler-based workload classification is also potentially vulnerable to adversarial workloads. A process deliberately crafting its resource usage patterns to mimic a latency-critical workload could receive elevated scheduling priority at the expense of legitimately critical tasks. While this threat is unlikely in most enterprise environments, it is a real concern in multi-tenant public cloud settings where tenants

are mutually untrusting. Hardening the profiler against adversarial inputs through anomaly detection on feature trajectories—for example using one-class SVMs or autoencoder-based outlier scoring applied to the 24-dimensional feature vector—is an important direction for future work.

Finally, the simulation-based evaluation does not fully replicate the complexity of real hardware. Physical systems introduce NUMA latency asymmetries, cache interference between co-scheduled processes, hardware interrupt handling delays, and thermal throttling behaviour that discrete-event simulation cannot faithfully model. A prototype implementation on real Linux kernel infrastructure—ideally using eBPF for safe kernel-space execution of the profiling and Q-value computation logic—is the natural and necessary next step before production deployment can be seriously considered. Early experiments with eBPF-based scheduling hooks suggest that the 320 KB memory budget and 2% overhead target are achievable on physical hardware, providing motivation to pursue this direction.

It is also worth reflecting on the broader implications of treating the OS scheduler as a learning system rather than a static policy engine. In traditional systems design, the scheduler is among the most carefully conserved components—changes to it require extensive regression testing and carry significant risk of introducing latency spikes or priority inversions. An adaptive scheduler that continuously updates its own policy raises legitimate concerns about stability guarantees and auditability. The logging mechanisms built into the Adaptive Queue Manager address the auditability concern in part, but formal verification of bounds on worst-case scheduling latency under the proposed policy—analogue to the analysis available for EDF under standard assumptions—remains an open and important research problem.

Beyond formal verification, the question of how to safely deploy a learning scheduler in a production environment that cannot tolerate service degradation during the learning phase deserves careful attention. One practical approach is shadow scheduling: running the RL agent in observation-only mode alongside the production scheduler, allowing it to build its Q-table and reach convergence before it is granted authority to make binding dispatch decisions. This canary deployment strategy, borrowed from software release engineering, provides a natural safety valve and could substantially reduce the risk associated with transitioning from conventional to intelligent scheduling in live systems.

VII. CONCLUSION

This paper presented a dynamic task scheduling and intelligent process control framework for AI-native operating systems. By integrating a reinforcement learning agent, a real-time workload profiler, an adaptive queue manager, and a hardware affinity controller into a unified, low-overhead scheduling architecture, the proposed system achieves consistent improvements in task completion time, CPU and GPU utilisation, and deadline adherence over conventional baselines including CFS^[1,2], EDF, and expert-

configured static priority scheduling. The quantitative gains—34.2% reduction in average completion time, 27.1% improvement in CPU utilisation, and a drop in deadline miss rate from 15.6% to 4.1%—are substantial enough to justify the modest additional complexity the approach introduces.

The core contribution is demonstrating that OS-level scheduling can be made adaptive and workload-aware without requiring application-level modifications, external cluster orchestration infrastructure, or unacceptable runtime overhead. The intelligence layer operates within a 2% CPU overhead budget and fits within 320 KB of kernel memory design constraints chosen to make the system feasible on resource-constrained embedded platforms as well as large-scale server deployments.

Several limitations of the current work motivate important directions for future research. The simulation-based evaluation provides strong motivation for prototype implementation on real kernel infrastructure using eBPF, which we identify as the primary immediate direction. Integration with federated learning mechanisms^[4] could allow multiple AI-native OS instances deployed across a cluster to share scheduling policy knowledge without centralising sensitive workload data, enabling faster adaptation to novel workload types. Coupling the scheduler with CPU frequency scaling and power management subsystems offers the opportunity for joint optimisation of performance and energy consumption—increasingly important as data center power costs rise alongside the intensity of AI workloads. Finally, extending the framework to handle hierarchical scheduling decisions across the full system stack from hardware interrupt handling to container orchestration^[8,9] represents an ambitious but important long-term goal for truly cognitive, self-managing computing infrastructure.

More broadly, the results of this study suggest that the boundary between the operating system and machine learning is dissolving. For decades, these two domains developed largely in isolation—OS researchers optimised schedulers for deterministic correctness and bounded worst-case behaviour, while ML researchers sought to maximise statistical performance on benchmark tasks. The framework presented here represents a concrete step toward their convergence: a production-oriented scheduling system that achieves measurable statistical performance improvements alongside the deterministic safety properties—bounded overhead, starvation prevention, and configurable fallback behaviour—that operating systems require. We believe this direction will become increasingly central to systems research as AI workloads continue to dominate computing infrastructure worldwide.

The open-source release of our simulation infrastructure, training workload traces, and Q-table initialisation files is planned to facilitate reproducibility and to encourage the community to benchmark alternative scheduling policies against the IDS baseline on a common platform. We also invite collaboration on the eBPF prototype implementation, which will require careful co-design between the kernel scheduling subsystem and the in-kernel ML inference

pipeline to meet the strict latency and memory constraints of production deployments.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [2] I. Molnar, “Modular Scheduler Core and Completely Fair Scheduler,” *Linux Kernel Mailing List*, Apr. 2007.
- [3] Z. Shen, S. Subbiah, X. Gu, and J. Wilkes, “CloudScale: Elastic Resource Scaling for Multi-Tenant Cloud Systems,” in *Proc. ACM SOCC, Cascais*, 2011, pp. 1–14.
- [4] R. Cortez et al., “Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms,” in *Proc. 26th SOSPP, Shanghai*, 2017, pp. 153–167.
- [5] C. Delimitrou and C. Kozyrakis, “Paragon: QoS-Aware Scheduling for Heterogeneous Datacenters,” in *Proc. 18th ASPLOS, Houston, TX*, 2013, pp. 77–88.
- [6] H. Mao et al., “Learning Scheduling Algorithms for Data Processing Clusters,” in *Proc. ACM SIGCOMM, Beijing*, 2019, pp. 270–288.
- [7] A. Mirhoseini et al., “Device Placement Optimization with Reinforcement Learning,” in *Proc. 34th ICML, Sydney*, 2017, pp. 2430–2439.
- [8] A. Verma et al., “Large-Scale Cluster Management at Google with Borg,” in *Proc. 10th EuroSys, Bordeaux*, 2015, pp. 1–17.
- [9] T. L. Rodeheffer et al., “Autopilot: Workload Autoscaling at Google,” in *Proc. 14th EuroSys, Dresden*, 2019, Art. no. 15.
- [10] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis,” in *Proc. ACM SOCC, San Jose, CA*, 2012, pp. 1–13.